

NewsMatrix Deployment & Configuration Guide

This document describes how to configure the local development environment and deploy the **NewsMatrix** application to production environments (e.g., AWS EC2).

1. Prerequisites

To run all components of NewsMatrix, verify the host has the following dependencies:

- **Python 3.10+** (to run the Backend API and Worker services).
 - **Node.js 18+ & npm** (to run the Frontend Vue 3 development server).
 - **PostgreSQL 14+** (with the **pgvector** extension installed).
 - **Redis 7+** (for caching).
 - **RabbitMQ 3+** (with the **rabbitmq_management** plugin enabled for the management UI).
 - **Supabase Account** (to retrieve API URL and keys for object storage).
 - **OpenAI API Key** (to calculate embeddings and query LLMs for the RAG assistant).
-

2. Local Development Environment Setup

2.1. Run Database and Services via Docker

To run PostgreSQL, Redis, and RabbitMQ quickly, you can run them using Docker:

- **Start Redis:**

```
docker run --name newsmatrix-redis -p 6379:6379 -d redis:7-alpine
```

- **Start RabbitMQ:**

```
docker run --name newsmatrix-rabbitmq -p 5672:5672 -p 15672:15672 -d rabbitmq:3-management
```

The management dashboard will be available at `http://localhost:15672` (default credentials: `guest / guest`).

- **Start PostgreSQL with pgvector:**

```
docker run --name newsmatrix-postgres -e POSTGRES_DB=newsmatrix -e POSTGRES_PASSWORD=secret -p 5432:5432 -d ankane/pgvector
```

2.2. Backend Installation

1. **Navigate to the backend directory and create a Python virtual environment:**

```
cd backend
python -m venv .venv
source .venv/bin/activate # On Windows: .venv\Scripts\activate
```

2. Install requirements:

```
pip install -r requirements.txt
```

3. Configure the Environment file (.env): Create a file named backend/.env with the following configuration:

```
# Database Configuration
SQLALCHEMY_DATABASE_URL=postgresql://postgres:secret@localhost:5432/newsmatrix

# Redis Configuration
REDIS_URL=redis://localhost:6379/0
CACHE_TTL_SECONDS=120
COMMENT_CACHE_TTL_SECONDS=300

# RabbitMQ Configuration
RABBITMQ_URL=amqp://guest:guest@localhost:5672/%2F
RABBITMQ_QUEUE_NAME=news_interactions

# Supabase Storage Configuration
SUPABASE_URL=https://your-project-id.supabase.co
SUPABASE_SERVICE_ROLE_KEY=your-supabase-service-role-key
SUPABASE_RAW_DATA_BUCKET=raw_data
SUPABASE_CHAT_IMAGES_BUCKET=chat-images

# OpenAI Config (RAG)
OPENAI_API_KEY=sk-proj-yourOpenAiApiKey

# Security
SECRET_KEY=your-super-secret-jwt-key-minimum-32-chars
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=60
```

4. Run database migrations:

```
alembic upgrade head
```

5. Start the API Server:

```
uvicorn api.main:app --reload --port 8000
```

Swagger API documentation will be served at <http://localhost:8000/docs>.

6. Start the background Worker: Open a separate terminal, activate the virtual environment, and run:

```
python -m api.worker
```

2.3. Frontend Installation

1. Navigate to the frontend directory:

```
cd frontend
```

2. Install dependencies:

```
npm install
```

3. Configure the Environment file (.env):

Create a file named frontend/.env pointing to the API URL:

```
VITE_API_URL=http://localhost:8000
```

4. Start the development server:

```
npm run dev
```

The client app will be available at `http://localhost:5173`.

3. Production Deployment Guide (AWS EC2)

When hosting the application on a single AWS EC2 instance, consider the following production configurations:

3.1. Configure LRU Eviction for Redis Cache

To prevent Redis from consuming all available RAM, configure memory caps and LRU policies in `/etc/redis/redis.conf`:

```
maxmemory 512mb
maxmemory-policy allkeys-lru
```

The `allkeys-lru` policy guarantees that old metrics drop out automatically when the memory cap is hit, preserving system stability.

3.2. Daemonize Backend Services

Configure FastAPI and Worker services to run under systemd unit files or launch them inside Docker container environments using `docker-compose.yml` to support auto-restart policies.

3.3. Reverse Proxy Configuration (Nginx)

Install Nginx to terminate SSL certificates (HTTPS) and proxy traffic:

- Proxy API traffic `/api/*` and `/assistant/*` requests to the local port `8000`.
- Serve Frontend build output (`dist` directory generated after running `npm run build`) as static assets.